Database System Principle - Relational Database Design Theory & Normal Forms

李旭东 **Li-Xudong**
LEEXUDONG@NANKAI.EDU.CN
NANKAI UNIVERSITY

OBJECTIVES

- Features of Good Relational Design
- Atomic Domains and First Normal Form
- Decomposition Using Functional Dependencies
- Functional Dependency Theory
- Algorithms for Functional Dependencies
- Decomposition Using Multivalued Dependencies
- Database-Design Process
- Modeling Temporal时间 Data

FEATURES OF GOOD RELATIONAL DESIGN

FEATURES OF GOOD RELATIONAL DESIGN

- Design Alternative:
  - Larger Schemas、Smaller Schemas

classroom(<u>*building*</u>, <u>*room_number*</u>, *capacity*)
department(<u>*dept_name*</u>, *building*, *budget*)
course(<u>*course_id*</u>, *title*, *dept_name*, *credits*)
instructor(<u>*ID*</u>, *name*, *dept_name*, *salary*)
section(<u>*course_id*</u>, <u>*sec_id*</u>, <u>*semester*</u>, <u>*year*</u>, *building*, *room_number*, *time_slot_id*)
teaches(<u>*ID*</u>, <u>*course_id*</u>, <u>*sec_id*</u>, <u>*semester*</u>, <u>*year*</u>)
student(<u>*ID*</u>, *name*, *dept_name*, *tot_cred*)
takes(<u>*ID*</u>, <u>*course_id*</u>, <u>*sec_id*</u>, <u>*semester*</u>, <u>*year*</u>, *grade*)
advisor(<u>*s_ID*</u>, *i_ID*)
time_slot(<u>*time_slot_id*</u>, <u>*day*</u>, <u>*start_time*</u>, *end_time*)
prereq(<u>*course_id*</u>, <u>*prereq_id*</u>)

COMBINE SCHEMAS? LARGE SCHEMAS

- **Suppose** we combine *instructor* and *department* into *inst_dept*
  - *(No connection to relationship set inst_dept)*

ID	*name*	*salary*	*dept_name*	*building*	*budget*
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

- Data redundancy数据冗余
- Inconsistency
- Insertion anomaly插入异常
- Deletion anomaly删除异常
- Update anomaly更新异常
- inability to represent information无法表示信息
  - Sum of budget

A COMBINED SCHEMA WITHOUT REPETITION

- Consider combining relations
  - *sec_class(sec_id, building, room_number)* and
  - *section(course_id, sec_id, semester, year)*

  into one relation
  - *section(course_id, sec_id, semester, year, building, room_number)*
- No repetition in this case

SMALLER SCHEMAS 1/2

- Suppose we had started with *inst_dept*
  - *(ID, name, salary, dept_name, building, budget)*
- How would we know to split up (**decompose**) it into *instructor* and *department*?
  - Write a rule "if there were a subschema (*dept_name, building, budget*), then *dept_name* would be a candidate key"

SMALLER SCHEMAS 2/2

- One Case:

  dept_name: building, budget
- In *inst_dept*, because *dept_name* is not a candidate key, the building and budget of a department may have to be repeated.
  - This indicates the need to decompose *inst_dept*

WHAT ABOUT SMALLER SCHEMAS? 1/3

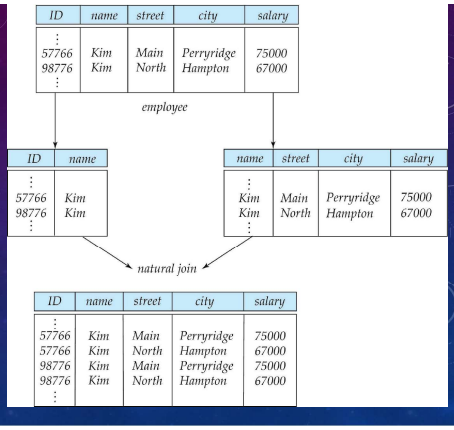
- Not all decompositions are good.
- Suppose we decompose

  employee(ID, name, street, city, salary) into

  employee1 (*ID*, *name*)

  employee2 (*name*, *street*, *city*, *salary*)

WHAT ABOUT SMALLER SCHEMAS? 2/3 : A LOSSY DECOMPOSITION



WHAT ABOUT SMALLER SCHEMAS? 3/3

- Not all decompositions are good.
- Suppose we decompose

  employee(ID, name, street, city, salary) into

  employee1 (*ID*, *name*)

  employee2 (*name*, *street*, *city*, *salary*)
- The above slide shows how we lose information -- we cannot reconstruct the original *employee* relation -- and so, this is a **lossy decomposition**有损分解.

EXAMPLE OF LOSSLESS-JOIN DECOMPOSITION

- **Lossless join decomposition**无损连接之分解
- Decomposition of *R = (A, B, C)*

  $R_1 = (A, B)$ $R_2 = (B, C)$

A	B	C
α	1	A
β	2	B

r

A	B
α	1
β	2

$\Pi_{A,B}(r)$

B	C
1	A
2	B

$\Pi_{B,C}(r)$

A	B	C
α	1	A
β	2	B

$\Pi_{A,B}(r) \bowtie \Pi_{B,C}(r)$

NORMAL FORM OF RELATIONAL SCHEMAL

FIRST NORMAL FORM

- Domain is **atomic原子的** if its elements are considered to be indivisible units
  - Examples of non-atomic domains:
    - Set of names, composite attributes
    - Address: street_city_state_zip

FIRST NORMAL FORM第一范式

- A relational schema R is in **first normal form** if the domains of all attributes of R are atomic

FIRST NORMAL FORM

- Atomicity原子性 is actually a property of how the elements of the domain are used.
  - Example: Strings would normally be considered indivisible
  - Suppose that students are given roll numbers which are strings of the form *CS2018012* or *EE1127*
    - If the first two characters are extracted to find the department, the domain of roll numbers is not atomic.
- Doing so is a bad idea: leads to encoding of information in application program rather than in the database.

FIRST NORMAL FORM

- Non-atomic values complicate复杂化 storage and encourage redundant (repeated) storage of data
  - Examples
    - Set of accounts stored with each customer, and set of owners stored with each account

EXAMPLE OF NORMAL FORM

- Does the following schema belong to 1NF?
- SLC(Sid, Sdept, Dloc, Cid, Cscore)
  - Sid: student id, Sdept: student's department
  - Cid:Course id, Dloc: location of department
  - Cscore: student's score of course

OUR GOAL :
DEVISE A THEORY FOR THE FOLLOWING WORK

- 1. Decide whether a particular relation *R* is in "good" form.
- 2. In the case that a relation *R* is not in "good" form, decompose it into a set of relations $\{R_1, R_2, ..., R_n\}$ such that
  - (1) each relation is in good form
  - (2) the decomposition is a **lossless-join decomposition**

OUR GOAL :
DEVISE A THEORY FOR THE FOLLOWING WORK

- Decide whether a particular relation *R* is in "good" form
  - Our theory is based on:
    - **functional dependencies**函数依赖
    - …

FUNCTIONAL DEPENDENCIES函数依赖

- Let *R* be a relation schema

  $\alpha \subseteq R$ *and* $\beta \subseteq R$
- The **functional dependency**

  $\alpha \rightarrow \beta$

  holds on(成立) *R* if and only if for any legal relations *r*(R), whenever any two tuples t_1 and t_2 of *r* agree on the attributes α, they also agree on the attributes β. That is,

  $t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$

FUNCTIONAL DEPENDENCIES (CONT.)

- Example: Consider *r*(A,*B*) with the following instance of *r.*

1	4
1	5
3	7

- On this instance, $A \rightarrow B$ does **NOT** hold, but $B \rightarrow A$ does hold.

FUNCTIONAL DEPENDENCIES: SUPERKEY V.S. CANDIDATE KEY

- *K* is a superkey for relation schema *R* if and only if $K \rightarrow R$
- *K* is a candidate key for *R* if and only if
  - $K \rightarrow R$, and
  - for no $\alpha \subset K$, $\alpha \rightarrow R$
- *For any candidate key K, if* $\alpha \in K$, *then* α is prime attribute (主属性), otherwise α is nonprime attribute (非主属性),

FUNCTIONAL DEPENDENCIES V.S. SUPERKEY

- Functional dependencies allow us to express constraints that cannot be expressed using superkeys. Consider the schema:

  inst_dept (*ID, name, salary, dept_name, building, budget*).

  We expect these functional dependencies to hold:

  dept_name→ *building*

  and *ID* → *building*

  but would not expect the following to hold:

  dept_name → *salary*

FUNCTIONAL DEPENDENCIES

- Constraints on the set of legal relations
- Require that the value for a certain set of attributes determines uniquely the value for another set of attributes
- A functional dependency is a generalization概化 of the notion of a *key*

USE OF FUNCTIONAL DEPENDENCIES 1/2

- We use functional dependencies to:
  - (1) test relations to see if(判定) they are legal under a given set of functional dependencies.
    - If a relation *r* is legal under a set *F* of functional dependencies, we say that *r* **satisfies(满足)** *F*.
  - (2) specify说明 constraints on the set of legal relations
    - We say that *F* **holds on** *R* if all legal relations on *R* satisfy the set of functional dependencies *F*.

USE OF FUNCTIONAL DEPENDENCIES 2/2

- Note:
  - A specific instance of a relation schema may satisfy a functional dependency even if the functional dependency does not hold on all legal instances.
  - For example
    - a specific instance of *instructor* may, by chance, satisfy *name* $\rightarrow$ *ID*

FUNCTIONAL DEPENDENCIES: TRIVIAL平凡的

- *A* functional dependency is **trivial** if it is satisfied by all instances of a relation
  - Example*:*
    - *ID, name* $\rightarrow$ *ID*
    - *name* $\rightarrow$ *name*
  - In general, $\alpha \rightarrow \beta$ is trivial if $\beta \subseteq \alpha$
- *Nontrivial非平凡的*

FUNCTIONAL DEPENDENCIES: FULL/PARTIAL FD

- Full FD 完全函数依赖
  - *IF* $X \rightarrow Y$, *and* $X' \not\rightarrow Y \quad (\forall X' \subset X)$
  - *THEN* $X \xrightarrow{F} Y$

Takes:
(S_ID, C_ID, score)

$(S\_ID, C\_ID) \xrightarrow{F} score$

- Partial FD 部分函数依赖

$X \xrightarrow{P} Y$

inst_dept (*ID, name, salary, dept_name, building, budget*)

$(ID, dept\_name) \xrightarrow{P} building$

FUNCTIONAL DEPENDENCIES: TRANSITIVE FD

- Given a set *F* of functional dependencies
  - If $A \rightarrow B$ and $B \rightarrow C$, and $B \not\subset A$, and $C \not\subset B$
  - then we can infer that $A \rightarrow C$
- we can call $A \rightarrow C$ *is a transitive FD (传递式函数依赖)*
- We also call $A \rightarrow C$ *is* logically implied(逻辑蕴涵) by *F*

CLOSURE(闭包) OF A SET OF FUNCTIONAL DEPENDENCIES

- Given a set *F* of functional dependencies, there are certain other functional dependencies that are logically implied by *F*.
  - For example: If $A \rightarrow B$ and $B \rightarrow C$, then we can infer that $A \rightarrow C$
- The set of **all** functional dependencies logically implied逻辑蕴涵 by *F* is the **closure** of *F*.
- We denote the *closure* of *F* by F^+.
- F^+ is a superset of *F*.

BOYCE-CODD NORMAL FORM: BCNF巴斯范式

A relation schema *R* is in BCNF with respect to a set *F* of functional dependencies if for all functional dependencies in F^+ of the form

$$\alpha \rightarrow \beta$$

where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following holds:

- (1) $\alpha \rightarrow \beta$ is trivial (i.e., $\beta \subseteq \alpha$)
- (2) α is a superkey for *R*

BOYCE-CODD NORMAL FORM: BCNF

Example schema *not* in BCNF:
instr_dept (*ID*, *name*, *salary*, *dept_name*, *building*, *budget*)
note: ID i.e. instructor id
because *dept_name*→ *building, budget*
holds on *instr_dept,* but *dept_name* is not a superkey

In fact: BCNF eliminates all redundancy that can be discovered based on functional dependencies

DECOMPOSING A SCHEMA INTO BCNF

- Suppose we have a schema *R* and a non-trivial dependency $\alpha \rightarrow \beta$ causes a violation(违规) of BCNF.

  We decompose *R* into:
  - R1: (α U β)
  - R2: (*R* - (β - α))

DECOMPOSING A SCHEMA INTO BCNF

- In our example

  instr_dept (*ID*, *name*, *salary*, *dept_name*, *building*, *budget*)
  - α = *dept_name*
  - β = *building, budget*

  and *inst_dept* is replaced by
  - (α U β) = (*dept_name*, *building, budget*)
  - (*R* - (β - α)) = (*ID*, *name, salary, dept_name*)

EXAMPLE OF NORMAL FORM

- Does the following schema belong to BCNF?
- SLC(Sid, Sdept, Dloc, Cid, Cscore)
  - Sid: student id, Cid:Course id, Dloc: Dept location

$$(\text{Sid}, \text{Cid}) \xrightarrow{F} Cscore$$

$$\text{Sid} \rightarrow \text{Sdept}, \quad (Sid, Cid) \xrightarrow{P} Sdept$$

$$Sid \rightarrow Dloc, \quad (Sid, Cid) \xrightarrow{P} Dloc,$$

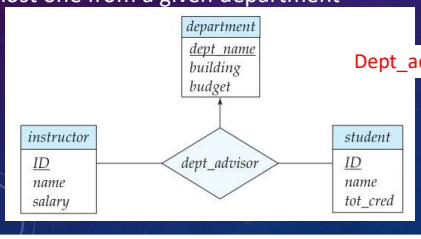
$$Sdept \rightarrow Dloc$$

DEPENDENCY PRESERVATION(保持依赖)

- *dependency preserving*
  - If it is sufficient to test only those dependencies on each individual relation of a decomposition in order to ensure that *all* functional dependencies hold, then that decomposition is *dependency preserving*.

EXAMPLE: DEPENDENCY PRESERVATION (1/3)

- Suppose: an instructor can be associated with only a single department and a student may have more than one advisor, but at most one from a given department



Dept_advisor (s_ID , i_ID , dept_name)

EXAMPLE: DEPENDENCY PRESERVATION (2/3)

Dept_advisor (s_ID , i_ID , dept_name)

- The following functional dependencies hold on dept_advisor:
  - (1) i_ID → dept_name
  - (2) s_ID , dept_name → i_ID

In fact: dept_advisor is not in BCNF because i_ID is not a superkey

EXAMPLE: DEPENDENCY PRESERVATION (3/3)

- Dept_advisor (s_ID , i_ID , dept_name)
  - (1) i_ID → dept_name
  - (2) s_ID , dept_name → i_ID
- If BCNF decomposition:
  - (i_ID , dept_name)
  - (s_ID , i_ID)
  - Both the above schemas

This design makes it computationally hard to enforce this functional dependency (2):
(2) s_ID , dept_name → i_ID
So it is **not dependency preserving**

BCNF & DEPENDENCY PRESERVATION

- Constraints, including functional dependencies, are ***costly*** to check in practice unless they pertain to only one relation
- It is not always possible to achieve both BCNF and dependency preservation
  - So, we consider a weaker normal form, known as *third normal form*.

THIRD NORMAL FORM第三范式

- A relation schema *R* is in **third normal form (3NF)** if for all:

$$\alpha \rightarrow \beta \text{ in } F^+$$

at least one of the following holds:

- (1) $\alpha \rightarrow \beta$ is trivial (i.e., $\beta \subseteq \alpha$)
- (2) α is a superkey for *R*
- **(3) Each attribute *A* in $\beta - \alpha$ is contained in a candidate key for *R*.**

(**NOTE**: each attribute may be in a different candidate key)

THIRD NORMAL FORM第三范式

- If a relation is in BCNF it is in 3NF
  - since in BCNF one of the first two conditions above must hold
- Third condition is a minimal relaxation of BCNF to ensure dependency preservation
  - Preserves function dependency of key attributes

THIRD NORMAL FORM第三范式: EXAMPLE

Dept_advisor (s_ID , i_ID , dept_name)

- The following functional dependencies hold on dept_advisor:
  - (1) i_ID → dept_name
  - (2) s_ID , dept_name → i_ID
- dept_advisor is not in BCNF because i_ID is not a superkey
- But dept_advisor is in third normal form
  - Because α=i_ID, *β=dept_name, so β- α= dept_name*
  - *Dept_name is contained in a candidate key*

EXAMPLE OF NORMAL FORM

- Does the following schema belong to 3NF?
- SLC(Sid, Sdept, Dloc, Cid, Cscore)
  - Sid: student id, Cid:Course id, Dloc: Dept location

$$(\text{Sid}, \text{Cid}) \xrightarrow{F} Cscore$$

$$\text{Sid} \rightarrow \text{Sdept}, \quad (Sid, Cid) \xrightarrow{P} Sdept$$

$$Sid \rightarrow \text{Dloc}, \quad (Sid, Cid) \xrightarrow{P} \text{Dloc},$$

$$Sdept \rightarrow \text{Dloc}$$

FUNCTIONAL-DEPENDENCY THEORY

FUNCTIONAL-DEPENDENCY THEORY

- We now consider the formal theory that tells us which functional dependencies are implied logically by a given set of functional dependencies.
- We then develop algorithms to generate lossless decompositions无损分解 into BCNF and 3NF
- We then develop algorithms to test if a decomposition is dependency-preserving依赖保持

FUNCTIONAL-DEPENDENCY THEORY - CLOSURE OF A SET OF FUNCTIONAL DEPENDENCIES

CLOSURE OF A SET OF FUNCTIONAL DEPENDENCIES

- Given a set *F* set of functional dependencies, there are certain other functional dependencies that are logically implied (逻辑蕴涵) by *F*.
  - For e.g.: If $A \rightarrow B$ and $B \rightarrow C$, then we can infer that $A \rightarrow C$
- The set of **all** functional dependencies logically implied by *F* is the **closure** of *F*.
- We denote the *closure* of *F* by F^+

CLOSURE OF A SET OF FUNCTIONAL DEPENDENCIES

- We can find F^+, the closure of F, by repeatedly applying **Armstrong's Axioms(公理)**
  - if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$ (**reflexivity自反律**)
  - if $\alpha \rightarrow \beta$, then $\gamma\,\alpha \rightarrow \gamma\,\beta$ (**augmentation增补律**)
  - if $\alpha \rightarrow \beta$, and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$ (**transitivity传递律**)
- **Armstrong's Axioms** are
  - **sound** 正确(generate only functional dependencies that actually hold)
  - **Complete完备** (generate all functional dependencies that hold).

CLOSURE OF A SET OF FUNCTIONAL DEPENDENCIES: EXAMPLE 1/2

- *R = (A, B, C, G, H, I)*
  F = { A → B, A → C, CG → H, CG → I, B → H}
- some members of F^+
  - $A \rightarrow H$: by transitivity from $A \rightarrow B$ *and* $B \rightarrow H$
  - $AG \rightarrow I$
    - by augmenting $A \rightarrow C$ with G, to get $AG \rightarrow CG$
      and then transitivity with $CG \rightarrow I$

CLOSURE OF A SET OF FUNCTIONAL DEPENDENCIES: EXAMPLE 2/2

- *R = (A, B, C, G, H, I)*
  F = { A → B, A → C, CG → H, CG → I, B → H}
- some members of F^+
  - $CG \rightarrow HI$
    - by augmenting $CG \rightarrow I$ to infer $CG \rightarrow CGI$,
      and augmenting of $CG \rightarrow H$ to infer $CGI \rightarrow HI$,
      and then transitivity

ADDITIONAL RULES OF ARMSTRONG'S AXIOMS

- If $\alpha \rightarrow \beta$ holds *and* $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta\gamma$ holds (**union合并律**)
- If $\alpha \rightarrow \beta\gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds (**decomposition分解律**)
- If $\alpha \rightarrow \beta$ holds *and* $\gamma\,\beta \rightarrow \delta$ holds, then $\alpha\,\gamma \rightarrow \delta$ holds (**pseudotransitivity伪传递律**)

The above rules can be inferred from Armstrong's axioms.

PROCEDURE FOR COMPUTING F^+

$F^+ = F$
repeat
  for each functional dependency *f* in F^+
    apply reflexivity自反律 and augmentation rules on *f*
    add the resulting functional dependencies to F^+
  for each pair of functional dependencies f_1 and f_2 in F^+
    if f_1 and f_2 can be combined using transitivity
    then add the resulting functional dependency to F^+
until F^+ does not change any further

FUNCTIONAL-DEPENDENCY THEORY - CLOSURE OF ATTRIBUTE SETS

CLOSURE OF ATTRIBUTE SETS属性集的闭包

- Given a set of attributes α, define ***the closure of α under F*** (denoted by α^+, or α_F^+) as the set of attributes that are functionally determined by α under *F*
- Algorithm to compute α^+, the closure of α under *F*

```
result := α;
while (changes to result) do
    for each β → γ in F do
      begin
        if β ⊆ result then result := result ∪ γ
      end
```


EXAMPLE OF ATTRIBUTE SET CLOSURE (1/2)

- *R = (A, B, C, G, H, I)*
- *F = {A → B, A → C , CG → H, CG → I, B → H}*
- *$(AG)^+$*
  1. ***result = AG***
  2. ***result = ABCG (A → C and A → B)***
  3. ***result = ABCGH (CG → H and CG ⊆ AGBC)***
  4. ***result = ABCGHI (CG → I and CG ⊆ AGBCH)***

EXAMPLE OF ATTRIBUTE SET CLOSURE (2/2)

- *R = (A, B, C, G, H, I)*
- *F = {A → B, A → C , CG → H, CG → I, B → H}*
- *$(AG)^+$= ABCGHI*
- Is *AG* a candidate key?
  1. Is AG a super key?
     1. Does *AG → R?* == Is $(AG)^+ \supseteq$ R
  2. Is any subset of AG a superkey?
     1. Does *A → R*? == Is $(A)^+ \supseteq$ R
     2. Does *G → R*? == Is $(G)^+ \supseteq$ R

USES OF ATTRIBUTE CLOSURE (1/2)

There are several uses of the attribute closure algorithm:

- (1) Testing for superkey:
  - To test if α is a superkey, we compute α^+, and check if α^+ contains all attributes of *R*.
- (2) Testing functional dependencies
  - To check if a functional dependency $\alpha \rightarrow \beta$ holds (or, in other words, is in F^+), just check if $\beta \subseteq \alpha^+$.
  - That is, we compute α^+ by using attribute closure, and then check if it contains β. Is a simple and cheap test, and very useful

USES OF ATTRIBUTE CLOSURE (2/2)

There are several uses of the attribute closure algorithm: (cont.,)

- (3) Computing closure of F
  - For each $\gamma \subseteq R$, we find the closure γ^+, and for each $S \subseteq \gamma^+$, we output a functional dependency $\gamma \rightarrow S$.

FUNCTIONAL-DEPENDENCY THEORY - CANONICAL COVER

CANONICAL COVER正则覆盖

- A canonical cover F_c of F is a "minimal" set of functional dependencies equivalent to F
  - having no redundant dependencies or redundant parts of dependencies

CANONICAL COVER正则覆盖

- Sets of functional dependencies may have redundant冗余 dependencies that can be inferred from the others
  - For example: $A \rightarrow C$ is redundant in: $\{A \rightarrow B,\ B \rightarrow C,\ A \rightarrow C\}$
  - Parts of a functional dependency may be redundant
    - E.g.: on RHS: $\{A \rightarrow B,\ B \rightarrow C,\ A \rightarrow CD\}$ can be simplified to $\{A \rightarrow B,\ B \rightarrow C,\ A \rightarrow D\}$
    - E.g.: on LHS: $\{A \rightarrow B,\ B \rightarrow C,\ AC \rightarrow D\}$ can be simplified to $\{A \rightarrow B,\ B \rightarrow C,\ A \rightarrow D\}$

EXTRANEOUS(无关的) ATTRIBUTES

- Consider a set F of functional dependencies and the functional dependency $\alpha \rightarrow \beta$ in F.
  - (1) Attribute A is **extraneous** in α if $A \in \alpha$ and F logically implies $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$.
  - (2) Attribute A is **extraneous** in β if $A \in \beta$ and the set of functional dependencies $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ logically implies逻辑蕴涵 F.
- *Note*: implication in the opposite direction is trivial in each of the cases above, since a "stronger" functional dependency always implies a weaker one

EXAMPLE: EXTRANEOUS(无关的) ATTRIBUTES

- Example: Given $F = \{A \rightarrow C,\ AB \rightarrow C\}$
  - B is extraneous in $AB \rightarrow C$ because $\{A \rightarrow C,\ AB \rightarrow C\}$ logically implies $A \rightarrow C$ (I.e. the result of dropping B from $AB \rightarrow C$).

EXAMPLE: EXTRANEOUS(无关的) ATTRIBUTES

- Example: Given $F = \{A \rightarrow C,\ AB \rightarrow CD\}$
  - C is extraneous in $AB \rightarrow CD$ since $AB \rightarrow C$ can be inferred even after deleting C

TESTING IF AN ATTRIBUTE IS EXTRANEOUS (1/2)

- Consider a set *F* of functional dependencies and the functional dependency $\alpha \rightarrow \beta$ in *F*.
- (1) To test if attribute $A \in \alpha$ is extraneous in α
  1. compute $(\{\alpha\} - A)^+$ using the dependencies in *F*
  2. check that $(\{\alpha\} - A)^+$ contains β; if it does, *A* is extraneous in α

TESTING IF AN ATTRIBUTE IS EXTRANEOUS (2/2)

- Consider a set *F* of functional dependencies and the functional dependency $\alpha \rightarrow \beta$ in *F*.

(cont.,)

- (2) To test if attribute $A \in \beta$ is extraneous in β
  1. compute α^+ using only the dependencies in $F' = (F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$,
  2. check that α^+ contains *A;* if it does, *A* is extraneous in β

F_C: CANONICAL COVER FOR F

- A **canonical cover**正则覆盖 for *F* is a set of dependencies F_c such that
  - *(1) F* logically implies all dependencies in F_c, and
  - *(2)* F_c logically implies all dependencies in *F,* and
  - (3) No functional dependency in F_c contains an extraneous attribute, and
  - (4) Each left side of functional dependency in F_c is **unique**.

F_C: CANONICAL COVER FOR F (CONT.)

- **To compute a canonical cover for *F***

F_c = F

repeat

Use the union rule to replace any dependencies in *F*
$\alpha_1 \rightarrow \beta_1$ and $\alpha_1 \rightarrow \beta_2$ with $\alpha_1 \rightarrow \beta_1 \beta_2$

Find a functional dependency $\alpha \rightarrow \beta$ with an extraneous attribute either in α or in β

/* Note: test for extraneous attributes done using F_c, not F*/

If an extraneous attribute is found, delete it from $\alpha \rightarrow \beta$

until F_c does not change

COMPUTING A CANONICAL COVER: EXAMPLE

- *R = (A, B, C) F = {A → BC, B → C, A → B, AB → C}*
- Combine *A → BC* and *A → B* into *A → BC*
  - Set is now *{A → BC, B → C, AB → C}*
- *A* is extraneous in *AB → C*
  - Check if the result of deleting A from *AB → C* is implied by the other dependencies
    - Yes: in fact, *B → C* is already present!
  - Set is now *{A → BC, B → C}*
  - …

COMPUTING A CANONICAL COVER: EXAMPLE (CONT.)

- *R = (A, B, C) F = {A → BC, B → C, A → B, AB → C}*
- *(cont.) Set is now {A → BC, B → C}*
- *C* is extraneous in *A → BC*
  - Check if *A → C* is logically implied by *A → B* and the other dependencies
    - Yes: using transitivity on *A → B and B →* C.
      - Can use attribute closure of *A* in more complex cases
- The canonical cover is: *A → B, B → C*

FUNCTIONAL-DEPENDENCY THEORY - LOSSLESS-JOIN DECOMPOSITION

LOSSLESS-JOIN DECOMPOSITION无损分解

- For the case of $R = (R_1, R_2)$, we require that for all possible relations r on schema R

$$r = \prod_{R1}(r) \bowtie \prod_{R2}(r)$$

LOSSLESS-JOIN DECOMPOSITION无损分解

- A decomposition of R into R_1 and R_2 is lossless join if at least one of the following dependencies is in F:
  - *(1)* $R_1 \cap R_2 \rightarrow R_1$
  - *(2)* $R_1 \cap R_2 \rightarrow R_2$
- The above functional dependencies are a sufficient condition for lossless join decomposition;
- the dependencies are a necessary condition only if all constraints are functional dependencies

判断无损分解的标准

LOSSLESS-JOIN DECOMPOSITION: EXAMPLE 1/2

- *R = (A, B, C) F = {A → B, B → C)*
  - Can be decomposed in two different ways
- *(1)* $R_1 = (A, B)$, $R_2 = (B, C)$
  - Lossless-join decomposition:

    $R_1 \cap R_2 = \{B\}$ and $B \rightarrow C$
  - Dependency preserving

LOSSLESS-JOIN DECOMPOSITION: EXAMPLE 2/2

- *R = (A, B, C) F = {A → B, B → C)*
  - Can be decomposed in two different ways
- *(2)* $R_1 = (A, B)$, $R_2 = (A, C)$
  - 2.1) Lossless-join decomposition:

    $R_1 \cap R_2 = \{A\}$ and $A \rightarrow B$
  - **2.2) Not dependency preserving**

    (cannot check $B \rightarrow C$ without computing $R_1 \bowtie R_2$)

FUNCTIONAL-DEPENDENCY THEORY - DEPENDENCY PRESERVATION

DEPENDENCY PRESERVATION依赖保持

- Let F_i be the set of dependencies F^+ that include only attributes in R_i
  - A decomposition is **dependency preserving**, **if**
    $(F_1 \cup F_2 \cup \ldots \cup F_n)^+ = F^+$
  - If it is not, then checking updates for violation of functional dependencies may require computing joins, which is expensive.

TESTING FOR DEPENDENCY PRESERVATION 1/2

- To check if a dependency $\alpha \rightarrow \beta$ is preserved in a decomposition of R into $R_1, R_2, \ldots, R_n$ we apply the following test (with attribute closure done with respect to F)
  - *1) result* = α
    while (changes to *result*) do
    for each R_i in the decomposition
    $t = (result \cap R_i)^+ \cap R_i$
    $result = result \cup t$
  - 2) If *result* contains all attributes in β, then the functional dependency $\alpha \rightarrow \beta$ is preserved.

TESTING FOR DEPENDENCY PRESERVATION 2/2

- We apply the test on all dependencies in F to check if a decomposition is dependency preserving
- This procedure takes polynomial time多项式时间, instead of the exponential time指数时间 required to compute F^+ and $(F_1 \cup F_2 \cup \ldots \cup F_n)^+$

TESTING FOR DEPENDENCY PRESERVATION: EXAMPLE

- $R = (A, B, C)$ $F = \{A \rightarrow B, B \rightarrow C\}$ Key = $\{A\}$
- R is not in BCNF
- Decomposition $R_1 = (A, B)$, $R_2 = (B, C)$
  - R_1 and R_2 in BCNF
  - Lossless-join decomposition
  - Dependency preserving

FUNCTIONAL-DEPENDENCY THEORY - ALGORITHMS FOR DECOMPOSITION

TESTING(判定) FOR BCNF

- To check if a non-trivial dependency $\alpha \rightarrow \beta$ causes a violation of BCNF
  1. compute α^+ (the attribute closure of α), and
  2. verify that it includes all attributes of R, that is, it is a superkey of R.

TESTING(判定) FOR BCNF

- **Simplified test**: To check if a relation schema *R* is in BCNF, it suffices足够 to check only the dependencies in the given set *F* for violation of BCNF, rather than checking all dependencies in F^+.
  - If none of the dependencies in *F* causes a violation of BCNF, then none of the dependencies in F^+ will cause a violation of BCNF either.

TESTING(判定) FOR BCNF

- However,
  - **simplified test using only *F* is incorrect when testing a relation in a decomposition of R**
  - Consider $R = (A, B, C, D, E)$, with $F = \{A \rightarrow B, BC \rightarrow D\}$
    - Decompose *R* into $R_1 = (A,B)$ and $R_2 = (A,C,D,E)$
    - Neither of the dependencies in *F* contain only attributes from (A,C,D,E) so we might be mislead into thinking R_2 satisfies BCNF.
    - In fact, dependency $AC \rightarrow D$ in F^+ shows R_2 is not in BCNF.

TESTING DECOMPOSITION FOR BCNF

- To check if a relation R_i in a decomposition of *R* is in BCNF,
  - Either test R_i for BCNF with respect to the **restriction限定** of F to R_i (that is, all FDs in F^+ that contain only attributes from R_i)
  - or use the original set of dependencies *F* that hold on *R*, but with the following test:
    - for every set of attributes $\alpha \subseteq R_i$, check that α^+ either includes no attribute of R_i- α, or includes all attributes of R_i.
  - If the condition is violated违背 by some $\alpha \rightarrow \beta$ in *F*, the dependency $\alpha \rightarrow (\alpha^+ - \alpha) \cap R_i$ can be shown to hold on R_i, and R_i violates BCNF.

 We use above dependency to decompose R_i

BCNF DECOMPOSITION ALGORITHM

result := {*R* }; *done* := false; compute F^+;
while (not *done)* **do**
  if (there is a schema R_i in *result* that is not in BCNF)
    then begin
      let $\alpha \rightarrow \beta$ be a nontrivial functional dependency that holds on R_i such that $\alpha \rightarrow R_i$ is not in F^+, and $\alpha \cap \beta = \varnothing$;
      result := $(result - R_i) \cup (R_i - \beta) \cup (\alpha, \beta)$;
    end
  else *done* := **true;**

each R_i is in BCNF, and decomposition is lossless-join

EXAMPLE OF BCNF DECOMPOSITION 1

- $R = (A, B, C)$ $F = \{A \rightarrow B, B \rightarrow C\}$ Key = $\{A\}$
- *R* is not in BCNF ($B \rightarrow C$ but *B* is not superkey)
- Decomposition
  - $R_1 = (B, C)$
  - $R_2 = (A,B)$

EXAMPLE OF BCNF DECOMPOSITION 2 1/3

- *class (course_id, title, dept_name, credits, sec_id, semester, year, building, room_number, capacity, time_slot_id)*
- Functional dependencies:
  - *course_id→ title, dept_name, credits*
  - *building, room_number→capacity*
  - *course_id, sec_id, semester, year→building, room_number, time_slot_id*
- A candidate key {*course_id, sec_id, semester, year*}.

EXAMPLE OF BCNF DECOMPOSITION 2 2/3

- BCNF Decomposition:
  - *course_id*→ *title*, *dept_name*, *credits* holds
    - but *course_id* is not a superkey.
  - We replace *class* by:
    - *course*(*course_id*, *title*, *dept_name*, *credits*)
    - *class-1* (*course_id*, *sec_id*, *semester*, *year*, *building*, *room_number*, *capacity*, *time_slot_id*)

EXAMPLE OF BCNF DECOMPOSITION 2 3/3

- *course* is in BCNF
  - How do we know this?
- *building*, *room_number*→*capacity* holds on *class-1*
  - but {*building*, *room_number*} is not a superkey for *class-1*.
  - We replace *class-1* by:
    - *classroom* (*building*, *room_number*, *capacity*)
    - *section* (*course_id*, *sec_id*, *semester*, *year*, *building*, *room_number*, *time_slot_id*)
- *classroom* and *section* are in BCNF.

BCNF AND DEPENDENCY PRESERVATION

It is not always possible to get a BCNF decomposition that is dependency preserving

- $R = (J, K, L)$ $F = \{JK \rightarrow L, L \rightarrow K\}$
  Two candidate keys = *JK* and *JL*
- *R* is not in BCNF
- Any decomposition of *R* will fail to preserve
  $JK \rightarrow L$
  This implies that testing for $JK \rightarrow L$ requires a join

FUNCTIONAL-DEPENDENCY THEORY - THIRD NORMAL FORM

THIRD NORMAL FORM: MOTIVATION

- There are some situations where
  - BCNF is not dependency preserving, and
  - efficient checking for FD violation on updates is important
- Solution: define a weaker normal form, called Third Normal Form (3NF)
  - Allows some redundancy (with resultant problems)
  - But functional dependencies can be checked on individual relations without computing a join.
  - There is always a lossless-join, dependency-preserving decomposition into 3NF.

THIRD NORMAL FORM第三范式

- A relation schema *R* is in **third normal form (3NF)** if for all:
  $\alpha \rightarrow \beta$ in F^+
  at least one of the following holds:
  - (1) $\alpha \rightarrow \beta$ is trivial (i.e., $\beta \subseteq \alpha$)
  - (2) α is a superkey for *R*
  - **(3) Each attribute *A* in $\beta - \alpha$ is contained in a candidate key for *R*.**
    (**NOTE**: each attribute may be in a different candidate key)

3NF EXAMPLE

- Relation *dept_advisor*:
  - *dept_advisor (s_ID, i_ID, dept_name)*
    $F = \{s\_ID, dept\_name \rightarrow i\_ID,\ i\_ID \rightarrow dept\_name\}$
  - Two candidate keys: *s_ID, dept_name,* and *i_ID, s_ID*
  - *R* is in 3NF
    - *s_ID, dept_name* $\rightarrow$ *i_ID*
      - *s_ID and dept_name* is a superkey
    - *i_ID* $\rightarrow$ *dept_name*
      - *dept_name* is contained in a candidate key

REDUNDANCY IN 3NF

- There is some redundancy in this schema
- Example of problems due to redundancy in 3NF
  - $R = (J, K, L)$ $F = \{JK \rightarrow L, L \rightarrow K\}$

J	L	K
j_1	l_1	k_1
j_2	l_1	k_1
j_3	l_1	k_1
null	l_2	k_2

- repetition of information (e.g., the relationship l_1, k_1)
  - (*i_ID, dept_name*)
- need to use null values (e.g., to represent the relationship l_2, k_2 where there is no corresponding value for *J*).
  - (*i_ID, dept_name*) if there is no separate relation mapping instructors to departments

TESTING判定 FOR 3NF

- Optimization: Need to check only FDs in *F*, need not check all FDs in F^+.
- Use attribute closure to check for each dependency $\alpha \rightarrow \beta$, if α is a superkey.
- If α is not a superkey, we have to verify if each attribute in β is contained in a candidate key of *R*
  - this test is rather more expensive, since it involve finding candidate keys
  - testing for 3NF has been shown to be NP-hard
  - Interestingly, decomposition into third normal form (described shortly) can be done in polynomial time

3NF DECOMPOSITION ALGORITHM 1/3

Let F_c be a canonical cover for *F;*
i := 0;
for each functional dependency $\alpha \rightarrow \beta$ in F_c **do**
then begin
i := *i* + 1;
$R_i := \alpha\ \beta$
end

3NF DECOMPOSITION ALGORITHM 2/3

if none of the schemas R_j, $1 \le j \le i$ contains a candidate key for *R*
then begin
i := *i* + 1;
R_i := any candidate key for *R;*
end

3NF DECOMPOSITION ALGORITHM 3/3

/* Optionally, remove redundant relations */
repeat
if any schema R_j is contained in another schema R_k
then /* delete R_j ***/**
$R_j = Ri$;
i=i-1;
until no more Rjs can be deleted
return (R_1, R_2, ..., R_i)

3NF DECOMPOSITION ALGORITHM

- Above algorithm ensures:
  - each relation schema R_i is in 3NF
  - decomposition is dependency preserving and lossless-join
  - Proof of correctness?

CORRECTNESS OF 3NF DECOMPOSITION ALGORITHM 1/5

- 3NF decomposition algorithm is dependency preserving (since there is a relation for every FD in F_c)
- Decomposition is lossless
  - A candidate key (C) is in one of the relations R_i in decomposition
  - Closure of candidate key under F_c must contain all attributes in R.
  - Follow the steps of attribute closure algorithm to show there is only one tuple in the join result for each tuple in R_i

CORRECTNESS OF 3NF DECOMPOSITION ALGORITHM 2/5

Claim: if a relation R_i is in the decomposition generated by the above algorithm, then R_i satisfies 3NF.

- Let R_i be generated from the dependency $\alpha \rightarrow \beta$
- Let $\gamma \rightarrow$ B be any non-trivial functional dependency on R_i. (We need only consider FDs whose right-hand side is a single attribute.)
- Now, B can be in either β or α but not in both. Consider each case separately.

CORRECTNESS OF 3NF DECOMPOSITION ALGORITHM 3/5

- Case 1: If B in β:
  - If γ is a superkey, the 2nd condition of 3NF is satisfied
  - Otherwise α must contain some attribute not in γ
  - Since $\gamma \rightarrow B$ is in F^+ it must be derivable from F_c, by using attribute closure on γ.
  - Attribute closure not have used $\alpha \rightarrow \beta$. If it had been used, α must be contained in the attribute closure of γ, which is not possible, since we assumed γ is not a superkey.

CORRECTNESS OF 3NF DECOMPOSITION ALGORITHM 4/5

- Case 1: If B in β:
  - (cont.,)
  - Now, using $\alpha \rightarrow$ (β- {B}) and $\gamma \rightarrow B$, we can derive $\alpha \rightarrow B$ (since $\gamma \subseteq \alpha \beta$, and B $\notin \gamma$ since $\gamma \rightarrow B$ is non-trivial)
  - Then, B is extraneous in the right-hand side of $\alpha \rightarrow \beta$; which is not possible since $\alpha \rightarrow \beta$ is in F_c.
  - Thus, if B is in β then γ must be a superkey, and the second condition of 3NF must be satisfied.

CORRECTNESS OF 3NF DECOMPOSITION ALGORITHM 5/5

- Case 2: B is in α.
  - Since α is a candidate key, the third alternative in the definition of 3NF is trivially satisfied.
  - In fact, we cannot show that γ is a superkey.
  - This shows exactly why the third alternative is present in the definition of 3NF.

Q.E.D.

3NF DECOMPOSITION: EXAMPLE 1/4

- Relation schema:

  cust_banker_branch = (*customer_id, employee_id, branch_name, type*)

- The functional dependencies for this relation schema are:
  1. *customer_id, employee_id* → *branch_name, type*
  2. *employee_id* → *branch_name*
  3. *customer_id, branch_name* → *employee_id*

3NF DECOMPOSITION: EXAMPLE 2/4

- We first compute a canonical cover
  - *branch_name* is extraneous in the r.h.s. of the 1st dependency
  - No other attribute is extraneous, so we get F_C =

    customer_id, employee_id → *type*
    employee_id → *branch_name*
    customer_id, branch_name → *employee_id*

3NF DECOMPOSITION: EXAMPLE 3/4

- The **for** loop generates following 3NF schema:

  (*customer_id, employee_id, type*)

  (*employee_id*, *branch_name*)

  (*customer_id, branch_name, employee_id)*
  - Observe that (*customer_id, employee_id, type*) contains a candidate key of the original schema, so no further relation schema needs be added

3NF DECOMPOSITION: EXAMPLE 4/4

- At end of for loop, detect and delete schemas, such as (*employee_id*, *branch_name*), which are subsets of other schemas
  - result will not depend on the order in which FDs are considered
- At last: The resultant simplified 3NF schema is

  (customer_id, employee_id, type)

  (customer_id, branch_name, employee_id)

COMPARISON OF BCNF AND 3NF

- It is always possible to decompose a relation into a set of relations that are in 3NF such that:
  - the decomposition is lossless
  - the dependencies are preserved
- It is always possible to decompose a relation into a set of relations that are in BCNF such that:
  - the decomposition is lossless
  - it may not be possible to preserve dependencies.

GOALS OF NORMALIZATION

GOALS OF NORMALIZATION 1/3

- Let *R* be a relation scheme with a set *F* of functional dependencies.
- Decide whether a relation scheme *R* is in "good" form.
- In the case that a relation scheme *R* is not in "good" form, decompose it into a set of relation scheme $\{R_1, R_2, ..., R_n\}$ such that
  - (1) each relation scheme is in good form
  - (2) the decomposition is a **lossless-join** decomposition
  - (3) Preferably, the decomposition should be **dependency preserving**

GOALS OF NORMALIZATION 2/3

- If we cannot achieve this, we accept one of
  - Solution 1:
    - Lack of dependency preservation
  - Solution 2:
    - Redundancy due to use of 3NF

GOALS OF NORMALIZATION 3/3

- Interestingly, SQL does not provide a direct way of specifying functional dependencies other than superkeys.
  - Can specify FDs using assertions断言, but they are expensive to test, (and currently not supported by any of the widely used databases!)
- Even if we had a dependency preserving decomposition, using SQL we would not be able to efficiently test a functional dependency whose left hand side is not a key.

HOW GOOD IS BCNF?

HOW GOOD IS BCNF?

- There are db schemas in BCNF that do not seem to be sufficiently normalized
- Consider a relation
  - An instructor can have many children and phone number, the phone numbers may be shared by multiple people
  - ID -> {child_name}
  - ID -> {phone}
- If we combine the above two schemas into one:

inst_info (ID, child_name, phone)

HOW GOOD IS BCNF?

- Consider a relation

inst_info (ID, child_name, phone)

This belongs to BCNF

- where an instructor may have more than one phone and can have multiple children

ID	*child_name*	*phone*
99999 99999 99999 99999	David David William Willian	512-555-1234 512-555-4321 512-555-1234 512-555-4321

HOW GOOD IS BCNF? (CONT.)

- There are no non-trivial functional dependencies and therefore the relation is in BCNF
- Insertion anomalies插入异常
  - i.e., if we add a phone 981-992-3443 to 99999, we need to add two tuples

    (99999, David, 981-992-3443)
    (99999, William, 981-992-3443)

HOW GOOD IS BCNF? (CONT.)

- Therefore, it is better to decompose *inst_info* into:

inst_child

ID	*child_name*
99999	David
99999	Willian

inst_phone

ID	*phone*
99999	512-555-1234
99999	512-555-4321

This suggests the need for higher normal forms, such as Fourth Normal Form (4NF)

MULTIVALUED DEPENDENCIES

MULTIVALUED DEPENDENCIES多值依赖

- Suppose we record names of children, and phone numbers for instructors:
  - *inst_child*(*ID*, *child_name*)
  - *inst_phone*(*ID*, *phone_number*)
- If we were to combine these schemas to get
  - *inst_info*(*ID*, *child_name*, *phone_number*)
  - Example data:
    (99999, David, 512-555-1234) (99999, David, 512-555-4321)
    (99999, William, 512-555-1234) (99999, William, 512-555-4321)
- This relation is in BCNF! Why?

MULTIVALUED DEPENDENCIES (MVDS) 1/2

- Let R be a relation schema and let $\alpha \subseteq R$ and $\beta \subseteq R$. The **multivalued dependency**

$$\alpha \twoheadrightarrow \beta$$

holds on R if in any legal relation $r(R)$, for all pairs for tuples t_1 and t_2 in r such that $t_1[\alpha] = t_2[\alpha]$, there exist tuples t_3 and t_4 in r such that:

$$t_1[\alpha] = t_2[\alpha] = t_3[\alpha] = t_4[\alpha]$$
$$t_3[\beta] = t_1[\beta]$$
$$t_3[R - \alpha - \beta] = t_2[R - \alpha - \beta]$$
$$t_4[\beta] = t_2[\beta]$$
$$t_4[R - \alpha - \beta] = t_1[R - \alpha - \beta]$$

MULTIVALUED DEPENDENCIES (MVDS) 2/2

- Tabular representation of $\alpha \twoheadrightarrow \beta$

	α	β	$R - \alpha - \beta$
t_1	$a_1 \dots a_i$	$a_{i+1} \dots a_j$	$a_{j+1} \dots a_n$
t_2	$a_1 \dots a_i$	$b_{i+1} \dots b_j$	$b_{j+1} \dots b_n$
t_3	$a_1 \dots a_i$	$a_{i+1} \dots a_j$	$b_{j+1} \dots b_n$
t_4	$a_1 \dots a_i$	$b_{i+1} \dots b_j$	$a_{j+1} \dots a_n$

MVDS EXAMPLE 1/2

- Let *R* be a relation schema with a set of attributes that are partitioned into 3 nonempty subsets.

  Y, Z, W

- We say that $Y \rightarrow\!\!\!\!\rightarrow Z$ (*Y* **multidetermines** *Z*) if and only if for all possible relations *r* (*R*)

  $< y_1, z_1, w_1 > \in r$ and $< y_1, z_2, w_2 > \in r$

  then

  $< y_1, z_1, w_2 > \in r$ and $< y_1, z_2, w_1 > \in r$

- Note that since the behavior of *Z* and *W* are identical it follows that $Y \rightarrow\!\!\!\!\rightarrow Z$ if $Y \rightarrow\!\!\!\!\rightarrow W$

MVDS EXAMPLE 2/2

- In our example:

  $ID \rightarrow\!\!\!\!\rightarrow$ *child_name*
  $ID \rightarrow\!\!\!\!\rightarrow$ *phone_number*

- The above formal definition is supposed to formalize the notion that given a particular value of *Y* (*ID*) it has associated with it a set of values of *Z (child_name)* and a set of values of *W (phone_number)*
  - these two sets are in some sense independent of each other.

USE OF MULTIVALUED DEPENDENCIES

- We use multivalued dependencies in two ways:
  1. To test relations to **determine** whether they are legal under a given set of functional and multivalued dependencies
  2. To specify **constraints** on the set of legal relations. We shall thus concern ourselves *only* with relations that satisfy a given set of functional and multivalued dependencies.
- If a relation *r* fails to satisfy a given multivalued dependency, we can construct a relations *r'* that does satisfy the multivalued dependency by adding tuples to *r.*

THEORY OF MVDS 1/2

- From the definition of multivalued dependency, we can derive the following rule: If $\alpha \rightarrow \beta$, then $\alpha \rightarrow\!\!\!\!\rightarrow \beta$

  That is, **every functional dependency is also a multivalued dependency**

- If $\alpha \rightarrow\!\!\!\!\rightarrow \beta$, then $\alpha \rightarrow\!\!\!\!\rightarrow R - \alpha - \beta$
- In fact
  - Functional dependency: equality-generating dependency
  - Multivalued dependency: tuple-generating dependency

函数依赖只是多值依赖的特例而已

THEORY OF MVDS 2/2

- The **closure** D^+ of *D* is the set of all functional and multivalued dependencies logically implied by *D*.
  - We can compute D^+ from *D*, using the formal definitions of functional dependencies and multivalued dependencies.
  - We can manage with such reasoning for very simple multivalued dependencies, which seem to be most common in practice
  - For complex dependencies, it is better to reason about sets of dependencies using a system of inference rules.

FOURTH NORMAL FORM

- A relation schema *R* is in **4NF** with respect to a set *D* of functional and multivalued dependencies if for all multivalued dependencies in D^+ of the form $\alpha \rightarrow\!\!\!\!\rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following hold:
  - (1) $\alpha \rightarrow\!\!\!\!\rightarrow \beta$ is trivial (i.e., $\beta \subseteq \alpha$ or $\alpha \cup \beta = R$)
  - (2) α is a superkey for schema *R*
- If a relation is in 4NF it is in BCNF

RESTRICTION OF MULTIVALUED DEPENDENCIES

- The restriction of D to R_i is the set D_i consisting of
  - All functional dependencies in D^+ that include only attributes of R_i
  - All multivalued dependencies of the form

    $\alpha \rightarrow\!\!\rightarrow (\beta \cap R_i)$

    where $\alpha \subseteq R_i$ and $\alpha \rightarrow\!\!\rightarrow \beta$ is in D^+

4NF DECOMPOSITION ALGORITHM

result: = {*R*}; *done* := false; *compute* D^+;
Let D_i denote the restriction of D^+ to R_i

while (**not** *done*)
if (there is a schema R_i in *result* that is not in 4NF) **then**
begin
let $\alpha \rightarrow\!\!\rightarrow \beta$ be a nontrivial multivalued dependency that holds on R_i such that $\alpha \rightarrow R_i$ is not in D_i, and $\alpha \cap \beta = \phi$;
result := (*result* - R_i) $\cup$ (R_i - β) $\cup$ (α, β);
end
else *done*:= true;

each R_i is in 4NF, and decomposition is lossless-join

EXAMPLE 1/2: 4NF DECOMPOSITION ALGORITHM

- $R = (A, B, C, G, H, I)$, $F = \{ A \rightarrow\!\!\rightarrow B,\ B \rightarrow\!\!\rightarrow HI,\ CG \rightarrow\!\!\rightarrow H \}$
- R is not in 4NF
  - since $A \rightarrow\!\!\rightarrow B$ and A is not a superkey for R
- Decomposition

  a) $R_1 = (A, B)$ (R_1 is in 4NF)

  b) $R_2 = (A, C, G, H, I)$ (R_2 is not in 4NF, decompose into R_3 and R_4)

EXAMPLE 2/2: 4NF DECOMPOSITION ALGORITHM

- $R = (A, B, C, G, H, I)$, $F = \{ A \rightarrow\!\!\rightarrow B,\ B \rightarrow\!\!\rightarrow HI,\ CG \rightarrow\!\!\rightarrow H \}$
- Decomposition

  ...

  c) $R_3 = (C, G, H)$ (R_3 is in 4NF)

  d) $R_4 = (A, C, G, I)$ (R_4 is not in 4NF, decompose into R_5 and R_6)
  - $A \rightarrow\!\!\rightarrow B$ and $B \rightarrow\!\!\rightarrow HI \Rightarrow A \rightarrow\!\!\rightarrow HI$, (MVD transitivity), and
  - and hence $A \rightarrow\!\!\rightarrow I$ *(MVD restriction to R_4)*

  e) $R_5 = (A, I)$ (R_5 is in 4NF)

  f) $R_6 = (A, C, G)$ (R_6 is in 4NF)

NORMAL FORMS

THE SECOND NORMAL FORM

- A relation schema R is in second normal form (2NF) if each attribute A in R meets one of the following criteria:
- (1) It appears in a candidate key
- (2) It is not partially dependent on a candidate key
  - 即非主属性完全函数依赖于任何一个候选码

EXAMPLE OF NORMAL FORM

- Does the following schema belong to 2NF?
- SLC(Sid, Sdept, Dloc, Cid, Cscore)
  - Sid: student id, Cno:Course id, Dloc: Dept location

$$(\text{Sid}, \text{Cid}) \xrightarrow{F} Cscore$$

$$\text{Sid} \rightarrow \text{Sdept}, \quad (Sid, Cid) \xrightarrow{P} Sdept$$

$$Sid \rightarrow \text{Dloc}, \quad (Sid, Cid) \xrightarrow{P} \text{Dloc},$$

$$Sdept \rightarrow \text{Dloc}$$

SUMMARY OF NORMAL FORM

1NF
↓ 消除**非主属性**对码的部分函数依赖
2NF
↓ 消除**非主属性**对码的传递函数依赖
3NF
↓ 消除**主属性**对码的部分和传递函数依赖
BCNF
↓ 消除非平凡且非函数依赖的**多值**依赖
4NF

FURTHER NORMAL FORMS

FURTHER NORMAL FORMS

- **Join dependencies连接依赖** generalize multivalued dependencies
  - lead to **project-join normal form (PJNF)** (also called **fifth normal form**)
- A class of even more general constraints, leads to a normal form called **domain-key normal form(DKNF)**.
- Problem with these generalized constraints: are hard to reason with, and no set of sound and complete set of inference rules exists.
- Hence rarely used

REVIEW : DEVISE A THEORY FOR THE FOLLOWING

- Decide whether a particular relation *R* is in "good" form.
- Our theory is based on:
  - **functional dependencies**函数依赖
  - **multivalued dependencies**多值依赖
  - Join dependencies连接依赖
  - project-join normal form投影-连接范式
  - domain-key normal form域-码范式

OVERALL DATABASE DESIGN PROCESS

OVERALL DATABASE DESIGN PROCESS

- We have assumed schema *R* is given
  - *1. R* could have been generated when converting E-R diagram to a set of tables.
  - *2. R* could have been a single relation containing *all* attributes that are of interest (called **universal relation单一关系**).
  - 3. Normalization breaks *R* into smaller relations.
  - *4. R* could have been the result of some ad hoc design of relations, which we then test/convert to normal form.

E-R MODEL AND NORMALIZATION

- When an E-R diagram is carefully designed, identifying all entities correctly, the tables generated from the E-R diagram should not need further normalization.
- However, in a real (imperfect) design, there can be functional dependencies from **non-key attributes of an entity to other attributes of the entity**
  - Example: an *employee* entity with attributes: *department_name* and *building*, and a functional dependency : *department_name* → *building*
  - Good design would have made department an entity
- Functional dependencies from non-key attributes of a relationship set possible, but rare --- most relationships are binary

NAMING OF ATTRIBUTES AND RELATIONSHIPS

- A desirable feature of a database design is the unique-role assumption, which means that each attribute name has a unique meaning in the database.
  - This prevents us from using the same attribute to mean different things in different schemas.

DENORMALIZATION去范式化 FOR PERFORMANCE

- May want to use non-normalized schema for performance
- For example, displaying ***prereqs*** along with ***course_id,*** and ***title*** requires join of *course* with *prereq*
- Alternative 1: Use denormalized relation containing attributes of *course* as well as *prereq* with all above attributes
  - faster lookup
  - extra space and extra execution time for updates
  - extra coding work for programmer and possibility of error in extra code

DENORMALIZATION FOR PERFORMANCE 2/2

- May want to use non-normalized schema for performance
- For example, displaying ***prereqs*** along with ***course_id,*** and ***title*** requires join of *course* with *prereq*
- Alternative 2: use a materialized view defined as: *course_prereq*
  - Benefits and drawbacks same as above, except no extra coding work for programmer and avoids possible errors

OTHER DESIGN ISSUES

- Some aspects of database design are not caught by normalization
- Examples of bad database design, to be avoided:

  Instead of *earnings (company_id, year, amount)*, use
  - *earnings_2004, earnings_2005, earnings_2006*, etc., all on the schema (*company_id, earnings*).
    - Above are in BCNF, but make querying across years difficult and needs new table each year

OTHER DESIGN ISSUES

- Some aspects of database design are not caught by normalization
- Examples of **bad database design**, to be avoided:

  Instead of *earnings* (*company_id, year, amount*), use
  - *company_year* (*company_id, earnings_2004, earnings_2005, earnings_2006*)
    - Also in BCNF, but also makes querying across years difficult and requires new attribute each year.
    - Is an example of a **crosstab**, where values for one attribute become column names
    - Used in spreadsheets, and in data analysis tools

MODELING TEMPORAL DATA

MODELING TEMPORAL DATA

- **Temporal data时态数据** have an association time interval during which the data are *valid*.
- A **snapshot快照** is the value of the data at a particular point in time
- Several proposals to extend ER model by adding valid time to
  - attributes, e.g., address of an instructor at different points in time
  - entities, e.g., time duration when a student entity exists
  - relationships, e.g., time during which an instructor was associated with a student as an advisor.
- But no accepted standard

MODELING TEMPORAL DATA

- Adding a temporal component results in functional dependencies like

  $ID \rightarrow street, city$

  not to hold, because the address varies over time
- A **temporal functional dependency** X → Y holds on schema *R* if the functional dependency X → Y holds on all snapshots for all legal instances r (*R*).

MODELING TEMPORAL DATA: EXAMPLE

- In practice, database designers may add start and end time attributes to relations
  - E.g., *course*(*course_id, course_title*) is replaced by

    course(*course_id, course_title, start, end*)
    - Constraint: no two tuples can have overlapping valid times
      - Hard to enforce efficiently
- Foreign key references may be to current version of data, or to data at a point in time
  - E.g., student transcript should refer to course information at the time the course was taken

SUMMARY

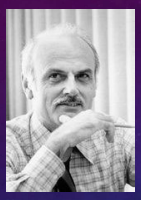
- Features of Good Relational Design
- Atomic Domains and First Normal Form
- Decomposition Using Functional Dependencies
- Functional Dependency Theory
- Algorithms for Functional Dependencies
- Decomposition Using Multivalued Dependencies
- Database-Design Process
- Modeling Temporal Data

BIBS

- Codd E F. A Relational Model for Large Shared Data Banks[J]. Communications of the Acm, 1970, 13(1):377--387. {1NF, 2NF, 3NF}
- Codd E F. Further Normalization of the Data Base Relational Model[J]. Data Base Systems, 1971, rj909. {BCNF}
- Biskup J, Dayal U, Bernstein P A. Synthesizing independent database schemas[C]// ACM SIGMOD International Conference on Management of Data, Boston, Massachusetts, May 30 - June. DBLP, 1979:143-151.
  - {a lossless dependency-preserving decomposition into 3NF}

BIBS

- Fagin R. Multivalued Dependencies And A New Normal Form For Relational Databases[J]. Acm Transactions on Database Systems, 1977, 2(3):262--278. {4NF}
- Armstrong W W. Dependency Structures of Data Base Relationships[C]// IFIP Congress. 1974:580-583. {Armstrong's axioms }
- Beeri C. A complete axiomatization for functional and multivalued dependencies in database relations[C]// ACM SIGMOD International Conference on Management of Data. ACM, 1977:47-61.
- ...



Edgar Frank Codd，1923－2003

William Ward Armstron

诗赞：数据库范式理论有感

- 盘古开天化万尘，物分类聚隐前因。
- 盲人摸象模型现，蚂蚁缘槐范式循。
- 函数关联出困顿，多值依赖解迷津。
- 登高渐览庐山面，回首方知此道真。

THANKS!

leexudong@nankai.edu.cn

Q&A?